

Executable Behavioral Design for Industrial Designers

Signature-First Mappings, Progressive Formalization, and a Typed Kernel for Product Logic

ZHU ZHEHAO
Research Design Draft

ABSTRACT

Industrial products increasingly embed sensing, temporal logic, software, and control, yet industrial designers still lack a behavioral design medium with the immediacy that CAD provides for geometry. Existing prototyping and visual-programming tools reduce implementation cost but often preserve engineering-oriented representations such as state transitions, execution steps, callbacks, or node-level computation. This paper proposes BDL, a Behavior Design Language whose primary artifacts are typed product relationships. A Mapping Block is created first by a semantic type signature, for example $?f : \text{Tilt} \rightarrow \text{Brightness}$, and may remain intentionally unresolved while the surrounding design is authored and checked. Formulas, curves, examples, temporal modifiers, hardware bindings, and verification obligations progressively refine the same artifact. BDL separates a designer-facing surface language from a kernel calculus with semantic quantity types, physical dimensions, declared clock domains, signals and events, scoped StateHandlers, deterministic action policies, typed holes, causality analysis, and a synchronous tick semantics. Temporal identity is declared rather than inferred, so that questions about differing update rates are raised while they are still semantic questions rather than at hardware binding. Core type soundness is kept separate from range, timing, and device-feasibility obligations. The paper also derives an interaction model that moves from broad functional intent to mapping detail, temporal/state detail, realization binding, and verification without requiring designers to adopt procedural implementation concepts. The central research hypothesis is that product behavior can become a design material: intuitive at the surface, but precise enough underneath to support early execution and validation.

CCS CONCEPTS Human-centered computing → User interface programming; Software and its engineering → Domain specific languages; Theory of computation → Type theory; Applied computing → Computer-aided design.

KEYWORDS industrial design, product behavior, executable design intent, type systems, reactive programming, typed holes, progressive formalization, clock domains, design tools

1 Introduction

Industrial design is increasingly concerned with products whose behavior is determined not only by geometry, materials, and mechanisms, but also by sensing, logic, timing, software, and networked control. A cup may infer that it has been lifted, a lamp may adapt to ambient conditions, a medical device may gate an action on multiple safety conditions, and a consumer robot may continuously map sensor estimates to actuator behavior. In such products, behavior is no longer a late implementation detail. It is part of the product concept.

Yet the dominant design media remain asymmetrical. Industrial designers have mature media for shape, layout, appearance, and physical assembly, while product behavior is usually externalized through prose, storyboards, flowcharts, state diagrams, interactive mock-ups, or ad hoc embedded code. The first four are easy to sketch but weak as executable specifications; the last is executable but forces the designer to adopt implementation-oriented concepts such as mutable variables, callbacks, polling loops, and device APIs. Existing physical prototyping systems such as Phidgets and d.tools substantially lowered the cost of building interactive prototypes [1], [2], but their behavioral representations still inherit important assumptions from programming and state-machine formalisms.

A concrete instance of the resulting cost appeared in a two-day introductory hardware workshop that one of the authors designed and taught for ten participants with no prior embedded experience. Two of the available sensors behaved in opposite ways: the ambient-light sensor reports larger values as illumination increases, while the distance sensor reports smaller values as the target recedes. Participants lost track of which was which, repeatedly and across the whole group. This pattern suggests a representational problem rather than merely a syntactic one. Whether a reading rises or falls with the quantity it measures is a stable fact about a device, and in the code they were writing there was nowhere to record it. It survived only as

a sign buried inside an expression and had to be reconstructed each time it was needed. The obstacle was not syntax. A piece of semantic information had no place to live.

This paper proposes a different boundary. The goal is not to make engineering representations merely easier for designers to use. The goal is to define a *native representation of product behavior for design itself*, while retaining enough formal structure for static checking, simulation, and eventual implementation.

We call the proposed system **BDL**, a Behavior Design Language. BDL is organized around a working hypothesis: when a behavioral relationship is first specified, *what kind of relationship should exist* is frequently settled before its exact implementation is. A designer may know that **Tilt influences Brightness** before deciding the transfer function; that **CupPickedUp** should activate a behavior before deciding which sensor and threshold detect pickup; or that a safety condition should suppress an actuator before choosing the device driver. Therefore the primary design artifact should be the *typed relationship*, not the procedure that computes it.

The central example is a Mapping Block. Instead of decomposing a design into procedural steps such as “read tilt,” “calculate brightness,” and “set the LED,” BDL represents one mapping:

$$?f : \text{Tilt} \rightarrow \text{Brightness} .$$

The mapping may exist before its body. A formula, curve, examples, or a fitted function can later be attached as a definition of the same block. Once a formula is supplied, for example

$$f(\theta) = \text{clamp}\left(0.2 + 0.8 \frac{\theta}{60^\circ}, 0, 1\right),$$

it inhabits the previously declared signature. This **signature-first authoring** model is not merely a user-interface convenience. It is the foundation of progressive formalization: an unresolved typed

mapping is a meaningful design artifact rather than a malformed program.

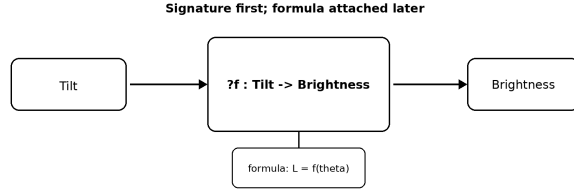


Figure 1: A signature-first Mapping Block. The flow graph contains one semantic relationship, $\text{Tilt} \rightarrow \text{Brightness}$; the formula is attached to the block rather than represented as an additional execution step.

BDL separates a designer-facing surface language from a small kernel calculus. The surface language supports named semantic types, typed holes, signal/event relationships, nested behavioral contexts, temporal modifiers, and external actions. The kernel tracks value types, physical dimensions, clock domains, state, and effect capabilities. It rejects implicit resampling, instantaneous cycles, untyped semantic conversions, non-total action handlers, and unresolved actuator conflicts. Range, real-time feasibility, and device limits are represented as separate validation obligations rather than being conflated with core type soundness.

This paper makes five contributions. First, it frames *designer agency over product behavior* as a representation problem rather than a simplified-programming problem. Second, it defines a signature-first surface language in which mappings, events, and state scopes can remain typed but intentionally incomplete. Third, it gives a kernel calculus and static semantics for quantities, clocks, reactive relationships, scoped state, action requests, and typed holes. Fourth, it describes an elaboration and typechecking architecture that preserves decidable core typing while generating separate engineering proof obligations. Fifth, it derives an interaction model in which designers progress from broad functional relationships to formulas, temporal semantics, device binding, and verification without being forced to confront all engineering detail at the start.

2 Design Goal: Reduce Semantic Translation, Not Key-strokes

2.1 The target user is not a programmer with fewer syntax skills

Many low-code and visual programming systems reduce textual syntax while preserving the underlying computational ontology: variables, assignments, loops, callbacks, functions, transitions, and scheduling. For software developers this can be convenient. For industrial designers, however, the dominant difficulty is often not syntax but *semantic translation*. The designer begins with a product statement such as “while the cup is held, brightness follows tilt” and must translate it into implementation machinery.

BDL therefore follows a stronger criterion: a surface primitive should be exposed only when it corresponds to a concept that is independently meaningful in the design task. A mutable accumulator used to count samples is generally not such a concept; “three pickup events within ten minutes” is. A polling loop is generally not; “while the product is held” is. A callback is not; “when the button is pressed” is.

The language should directly expose the following concepts: semantic properties, continuous signals, discrete events, typed mappings, conditions, behavioral contexts, temporal relations, actions, priorities, and safety constraints. By default it should hide program counters, threads, callbacks, continuations, clock variables, effect rows, SSA values, and bus transactions. These may remain inspectable in

an expert or debugging view, but they are not the primary design medium.

2.2 A flow graph is a dependency view, not a program counter

BDL uses a flow-like canvas because causal and functional paths are useful visual structures. The arrows do **not** mean “execute the left node and then the right node.” They mean that the target relationship depends on the source relationship or value.

Consider the procedural decomposition:

```

[Cup picked up]
|
[Read tilt]
|
[Compute L = f(theta)]
|
[Set LED brightness]
  
```

This decomposition is rejected as a primary design representation because the “read,” “compute,” and “set” nodes are artifacts of an execution model. The corresponding BDL view is a Held context containing a mapping from Tilt to Brightness . The formula is a property of that mapping, and the realization layer later binds Brightness to a physical light output.

The distinction matters because it changes what the designer edits. In a procedure-centric editor, changing an implementation may require rewriting steps and intermediate state. In BDL, changing a transfer function edits the definition of one relationship while preserving the surrounding product logic.

2.3 Cognitive budget

The surface language must remain intentionally small. Each additional primitive has a cost in learnability, visibility, consistency, and error-proneness, all familiar concerns in the Cognitive Dimensions tradition [3]. BDL therefore uses a *cognitive budget*: a new visible construct is justified only when it captures a distinct design concept that cannot be expressed cleanly as a property of an existing construct.

This yields several design rules. Formulas are attached to mappings rather than represented as separate nodes. Time is expressed through modifiers such as *for*, *after*, *while*, and *until*, not through explicit timer variables. Local history is exposed through semantic operators such as *previous*, *count*, and *since*, not arbitrary mutable cells. Device arbitration is surfaced only when multiple design intentions genuinely compete for the same actuator.

2.4 Progressive disclosure is a semantic property

A design tool can be visually simple and still force premature decisions. BDL instead treats progressive disclosure as part of the language semantics. A Level-0 model may contain a typed hole whose implementation is intentionally unresolved. Later levels may refine the same artifact with mathematical properties, temporal behavior, device bindings, and engineering constraints.

We distinguish the following authoring levels:

1. **Intent sketch.** Named product properties, events, contexts, actions, and rough relationships.
2. **Typed relationship.** Domain/codomain signatures and event payloads are fixed; mappings may remain holes.
3. **Behavioral definition.** Formulas, temporal modifiers, and local state semantics are supplied.
4. **Realization binding.** Abstract properties are bound to sensors, actuators, clocks, and device capabilities.
5. **Verification.** The system checks static typing and selected feasibility obligations.

The tool must never force a Level-1 decision to provide Level-4 details unless those details are semantically necessary to continue.

3 Surface Language

3.1 Semantic properties and signatures

The first-class visual object in BDL is a *semantic property*, not a raw scalar. Examples include *Tilt*, *Brightness*, *Temperature*, *CupContact*, and *MotorAngle*. Each semantic property has a representation type and may carry units, ranges, and documentation, but its name itself is significant. *Tilt* and *MotorAngle* may share an angular dimension without being interchangeable design concepts.

A Mapping Block is created from a signature

$$m : (A_1, \dots, A_n) \rightarrow B.$$

At creation time the implementation may be an unresolved metavariable:

$$?m : (A_1, \dots, A_n) \rightarrow B.$$

This is a valid partial artifact. The editor can already reject wires with incompatible domains, propagate semantic types downstream, show documentation for the intended relationship, and record additional properties such as monotonicity or continuity.

This design deliberately exploits the information density of a function type. *Real* $\rightarrow$ *Real* reveals little. *Tilt* $\rightarrow$ *Brightness* already communicates much of the design intent. More refined signatures can add dimensions and representation constraints without forcing those details into the main canvas.

3.2 Mapping Blocks

A Mapping Block has the surface record

$$\text{Mapping} = \{id, name, \overline{x : A, B}, \\ definitions : Def^*, \\ active : Index?, \\ properties : Prop^*\}.$$

A definition may be a mathematical expression, a piecewise curve, a set of input-output examples to be fitted, or a reference to an external component. All definition forms elaborate to the same kernel function boundary, so different authoring styles do not fragment the semantic model.

Three consequences follow from separating identity from definition.

Identity belongs to the signature. The pair $(id, name)$ is stable across every change to *definitions*. *id* is the internal referent; *name* is a mutable display name, so renaming is a refactoring that updates all reference sites rather than a textual edit. Because a mapping can be named before it can be computed, the rest of the model may refer to it, compose with it, and state properties about it while it is still undefined. Declared properties such as monotonicity attach to the name, not to any particular definition.

The unresolved state is a value of the same record. A typed hole is *active* = None, not a separate kind of object. Attaching a definition and detaching one are therefore both ordinary edits, and detaching is meaningful: it retracts an implementation while retaining the claim that the relationship exists.

Several candidate definitions may coexist. A formula, a fitted curve, and a reference to a supplied component can be attached to the same signature and compared, with exactly one *active* and the remainder archived. This supports the exploration of behavioral alternatives without duplicating the surrounding product logic. Attachment supplies an implementation; it never participates in dis-

patch, and a model with several simultaneously live definitions for one signature is rejected.

Nothing in this record imposes an authoring order. Declaring a signature and attaching a formula may be a single action, exactly as a type signature and an equation are written together in a functional language. What the record guarantees is that stopping between the two costs nothing.

For signal-valued ports, a pure mapping is lifted pointwise. If

$$f : A \rightarrow B,$$

then in domain d the reactive lifting has type

$$\text{map}_d(f) : \text{Signal } [d]A \rightarrow \text{Signal } [d]B.$$

The designer need not see this lifting. It is an elaboration rule implied by connecting a signal-valued property to a Mapping Block.

3.3 Signals and events

BDL distinguishes time-varying values from discrete occurrences, following a long line of reactive-language work [4], [5].

A *Signal*<*T*> denotes a value available at each tick of a clock. Examples are tilt angle, temperature, or estimated distance. An *Event*<*T*> denotes an occurrence that may or may not be present at a tick, optionally carrying a payload. Examples are “button pressed,” “pickup detected,” or “overheat warning.”

This distinction removes common hidden state. A rising-edge operator can convert a Boolean signal into an event without requiring the designer to create a *previous_state* variable. Conversely, converting an event into a held signal requires an explicit memory operator because doing so changes temporal semantics.

3.4 Time as a modifier, not a waiting instruction

The surface language avoids *wait* (300 ms) as a primary construct because *wait* suggests a suspended sequential thread. Instead, BDL uses temporal modifiers that describe relationships:

- p for 300 ms;
- after e by 2 s;
- while p;
- until e;
- within 5 s;
- since e;
- once;
- every 1 s.

These modifiers elaborate to finite-state stream transducers. The designer expresses a temporal property; the kernel carries the required state.

3.5 StateHandler as a behavioral scope

A *StateHandler* is not a program-counter location. Its surface meaning is:

Within this product context, these behavioral relationships are active.

A handler may be activated by an event, a Boolean condition, or a scoped interval. It contains a local flow graph, nested handlers, and optional policies for external actions. For example, a *Held* handler may be active while the cup is not in contact with the table. Inside it, a *Drinking* sub-handler may activate when tilt exceeds a semantic threshold.

Nested handlers form a tree, avoiding the need to flatten every orthogonal concern into a Cartesian-product state machine. BDL still borrows the proven value of hierarchical state models [6], but the

surface metaphor is a *context containing behavior*, not a transition diagram that the user must manually maintain.

3.6 Actions and effect requests

Actions are the only surface constructs that request a change to the external world. Examples include `SetBrightness`, `MoveMotor`, `PlaySound`, and `SendMessage`. Sensor values are signals; time is temporal structure; neither is modeled as an arbitrary side effect.

The design is inspired by the separation between effectful operations and their interpretation in algebraic-effect systems [7], but the first BDL kernel intentionally implements a smaller request-and-policy model rather than claiming a fully general algebraic-effect calculus. A `StateHandler` can allow, suppress, transform, cap, prioritize, or arbitrate action requests. This is sufficient for important design concerns such as low-power mode, safety overrides, and mutually exclusive actuator access while keeping continuations and generic effect handlers out of the surface language.

3.7 Typed holes as design artifacts

Typed holes are essential to BDL because incompleteness is normal during design. Work on typed structure editing, notably `Hazelnut`, shows that statically meaningful incomplete programs can be treated rigorously [8]. BDL adapts the idea to product behavior: an unresolved mapping is not a syntax error and not necessarily an “unfinished program.” It can be a deliberate statement that a relationship exists while its concrete realization remains open.

A hole is always constrained by an expected type. The surface editor may therefore show

```
?pickupDetector :  
  (Contact, Acceleration) -> Event<PickedUp>
```

while allowing the rest of the model to be authored and checked around it.

A BDL hole is *named*. The identifier is carried by the signature rather than by the position at which the hole occurs, so `?pickupDetector` is a referent: other parts of the model may consume it, compose with it, and carry declared obligations about it while nothing computes it. The name is also design information in its own right. For example, the name `?comfortCurve`, together with the signature `Posture → Comfort`, communicates considerably more than `?f` with the same type, in the same way that `Tilt → Brightness` communicates more than `Real → Real`. Names should therefore be shown prominently, be freely renameable, and admit documentation, rather than being treated as internal identifiers.

This is a minor but real divergence from structure editors in the `Hazelnut` tradition, where holes are typically positional. Persisting a hole across a long design process, and discussing it with collaborators, requires that it can be referred to.

3.8 Supplied computation blocks

Some product behavior is genuinely easier to state as code than as a diagram. Recursive filters, Kalman estimators, spectral transforms, and self-tuning controllers are not clarified by being decomposed into wires and formulas. BDL therefore admits computation blocks written in the host language and linked into the generated implementation.

This is not an escape hatch, and framing it as one misdescribes the workflow it supports. It is the interface across which an engineer supplies a designer with a capability. Because authoring is signature-first, the request can precede the implementation: the designer places

```
?smooth : Signal<d, Distance> -> Signal<d, Distance>
```

and the hole’s type is the specification the engineer works against. The same interface is used by the standard library that ships with

the system, deliberately, so that first-party components cannot rely on facilities denied to third-party ones.

A supplied block is either a pure function or a transducer of the form given for temporal primitives. It may not issue actions. Permitting device access inside supplied code would defeat effect-row accumulation and actuator-conflict analysis, which is the one boundary that must not be crossed.

What the kernel can derive for a native mapping must instead be declared for a supplied one: determinism, totality on well-typed inputs, absence of hidden effects, output range, any properties relied upon downstream such as monotonicity, worst-case state size, and worst-case execution time. Blocks placed inside a `StateHandler` must additionally support reset, since local temporal state is reset on entry; a block that cannot be reset may only appear at the top level. A block must also declare the update rate it was designed for, because a recursive filter transplanted from one rate to another does not fail, it silently acquires a different cutoff frequency. This last condition is checked against the bound rate of the block’s domain, and is the most common error in this class that can be caught statically.

These declarations enter Φ rather than the core type system, and the mechanism by which each is discharged is recorded with it.

4 Kernel Calculus

This section defines the small semantic core to which all surface constructs elaborate. The aim is not to encode every engineering detail in the type system. The kernel instead provides a decidable static foundation for semantic types, physical dimensions, clocks, causality, scoped state, and action capabilities; harder engineering properties are emitted as separate validation obligations.

4.1 Time, domains, and clocks

BDL separates *temporal identity*, which is fixed while the design is authored, from *temporal rate*, which is fixed only at realization binding.

4.1.1 Domains are declared, not inferred

A **clock domain** is a named declaration:

```
domain interaction  
domain ambient
```

A domain carries no rate at design time. It denotes a set of signals that are updated together, which is a product-level statement a designer can make before any hardware has been chosen. Reactive types are indexed by a domain name rather than by a clock variable:

$$\text{Signal } [d] \tau, \quad \text{Event } [d] \tau.$$

Domain equality is **syntactic**. There are no domain metavariables, no unification, and no domain inference. Clock domains are introduced only by explicit declarations (or by rank-1 domain parameters in reusable component definitions). A `StateHandler` does **not** create a new clock domain: it gates behavior within its enclosing domain while preserving the domain index of every signal and event inside the scope.

An implicit domain `main` exists in every model. A design that never needs to distinguish update rates never mentions a domain at all.

This is a deliberate departure from the synchronous-language tradition, in which clocks are recovered by a dedicated clock calculus [9]. BDL adopts clocks-as-types and rejects clocks-as-inference; the argument is developed in the discussion of nominal identity.

4.1.2 Binding assigns rates

Let physical time be

$$T = \mathbb{R}_{\geq 0}.$$

At realization binding each domain d receives a rate, that is, a strictly increasing sequence

$$\kappa_d : \mathbb{N} \rightarrow \mathbb{T}, \quad \forall n. \kappa_d(n+1) > \kappa_d(n).$$

For value type τ ,

$$\text{Signal } [d] \tau = \mathbb{N} \rightarrow \tau$$

and

$$\text{Event } [d] \tau = \mathbb{N} \rightarrow \text{Option } \tau.$$

For a finite set of bound domains, BDL also assumes that the union of their tick times is **locally finite**: every bounded physical-time interval contains only finitely many domain ticks. The distinct physical instants can therefore be enumerated by a strictly increasing global schedule

$$\kappa_G : \mathbb{N} \rightarrow \mathbb{T},$$

whose range is $\bigcup_d \text{range } (\kappa_d)$. At global step g , the set of domains that fire is

$$\text{Fire } (g) = \{d \mid \exists i. \kappa_d(i) = \kappa_G(g)\}.$$

The global schedule is an execution device, not a new domain visible to the designer. It exists only to give a precise order to multi-domain runtime steps and to make simultaneous physical timestamps explicit.

This discrete logical-time model does not claim that the physical world is discrete. It defines the boundary after sensing and before actuation at which the BDL runtime reasons. A continuously varying physical quantity enters the kernel through an explicitly bound sampling interface.

Domain identity is part of the reactive type. Signals in different domains cannot be combined by ordinary Mapping Blocks, and any cross-domain relationship must contain an explicit synchronization node.

4.1.3 Why identity precedes rate

Under clock inference the temporal structure of a Level-2 model is an unsolved constraint set. A clock metavariable denotes nothing until unification completes, so such a model is not yet correct; it is merely not yet solved. A rate conflict introduced at binding time then appears as a unification failure whose reported location is determined by traversal order rather than by design intent, and a single binding action can invalidate a whole graph at once. This is precisely the late, delocalized failure that the layered authoring model is intended to avoid.

Under declared domains the temporal structure of a Level-2 model is complete and determinate, merely abstract. `ambient` is a meaningful design object before any sensor is chosen, and binding refines it with a rate rather than resolving it. Cross-domain questions are therefore raised while the designer still holds the semantic information needed to answer them and does not yet need the engineering information they lack.

The commitment is the same one made for typed holes. A hole has no synthesis rule and cannot invent a type; likewise the checker cannot invent a domain. Temporal structure is authored, not recovered.

4.1.4 Domain polymorphism, and its limit

Declared domains give up principal typing. A reusable component such as a debounce filter would otherwise have to be duplicated per domain, so component definitions may be quantified:

$$\text{debounce} : \forall \delta. \text{Signal } [\delta] \text{ Bool} \rightarrow \text{Signal } [\delta] \text{ Bool} .$$

Quantification is prenex and rank-1, and instantiation is substitution of a domain name at the use site. There is deliberately no solving: the system supports domain-polymorphic *definitions*, not domain *inference*. This is the same arrangement as region annotation in languages with explicit lifetimes, where the annotation is written, locally elided, and never globally solved.

Two further consequences are accepted. Two domains that are eventually bound to the same rate still require an explicit synchronization node between them. Because cross-domain synchronization is a causal boundary, equal rates do **not** make that node observationally identical to an intra-domain wire: the target sees the last value committed strictly before its tick. Binding may therefore suggest merging the two domains if the designer actually intends zero-latency synchronous behavior, but the checker will never perform that merge implicitly. And a mid-design decision that a quantity belongs in a different domain requires editing a declaration and revisiting the cross-domain points it creates, where inference would have silently re-solved. The friction is proportionate to the design change it reflects.

4.2 Dimensions and semantic types

Let m be the number of chosen base physical dimensions and let

$$\mathbb{D} = \mathbb{Z}^m.$$

A physical quantity with dimension vector d has representation type

$$Q[d].$$

The standard dimensional rules apply:

$$Q[d] + Q[d] \rightarrow Q[d],$$

$$Q[d_1] \times Q[d_2] \rightarrow Q[d_1 + d_2],$$

and

$$Q[d_1] / Q[d_2] \rightarrow Q[d_1 - d_2].$$

4.2.1 Units live at the boundary, not in the kernel

Dimensions are part of the kernel type; *units* are not. The kernel works in one canonical unit per dimension, and a unit is surface metadata used for display and for realization binding. Two ports whose semantic types agree but whose declared units differ are connected by a conversion inserted at that boundary, since same-dimension unit change is multiplication by a positive scalar and therefore invertible and composable. Keeping units out of the kernel avoids conversion nodes accumulating along a path and avoids repeated round-tripping of representations.

Two cases are excluded from this convenience. Affine units, such as degrees Celsius against kelvin, are not scalar multiples, and the same unit label behaves differently on absolute values and on differences: a temperature of 20 °C is 293.15 K, whereas a temperature difference of 20 °C is 20 K. BDL does not resolve this dynamically. `Temperature` and `TemperatureDelta` are distinct semantic types, the first admitting affine conversion and not addition, the second admitting linear conversion and addition. Gauge and absolute pressure follow the same pattern. Conversions that lose precision under a bound representation, such as inches to millimetres in fixed point, are not rejected; they emit a validation obligation.

4.2.2 Semantic types

BDL additionally supports *nominal semantic types*

$$\text{Sem } [n, d],$$

where n is a semantic name and d is its physical dimension. Their representation can be accessed inside a mapping through an explicit kernel projection rep .

Dimensional agreement is therefore necessary but not sufficient for a connection. A standard example is torque versus energy: both have dimension $M L^2 T^{-2}$, yet they denote different physical concepts and are not interchangeable merely because their dimensions agree. Under nominal typing Torque and Energy are distinct, and moving between them requires a stated relationship. The same holds for Tilt and MotorAngle, which may both carry the physical dimension Angle while denoting unrelated design concepts.

4.2.3 Three kinds of dimensionlessness

A single dimension vector of zero conflates three situations that behave differently, so BDL distinguishes them.

Genuinely dimensionless quantities have $d = 0$ and participate fully in the dimensional rules: ratios, gains, probabilities, and normalized quantities such as Brightness. Discrete event counts are ordinary integers rather than physical quantities. Scaling relies on $Q[0] \times Q[d] \rightarrow Q[d]$.

Quantities that appear dimensionless but are not must carry their dimension. Frequency is T^{-1} ; when plane angle is retained as a design dimension, angular velocity is $\text{Angle} \cdot T^{-1}$; acceleration is $L \cdot T^{-2}$. Recording these as dimensionless collapses distinctions that the design and the checker may need later.

Non-physical semantic properties are written $\text{Sem } [n, \perp]$ and have no projection into Q . Comfort, Urgency, Mode, and SoundId are design concepts with no dimensional content. Encoding them as $Q[0]$ would admit expressions such as a comfort multiplied by a gain, which pass dimensional checking and are meaningless. A value of type $\text{Sem } [n, \perp]$ may be mapped, compared, and stored, but not entered into dimensional arithmetic.

This distribution has a consequence for where checking pressure falls. In the target domain most quantities visible at the intent, signature, and mapping-detail layers are dimensionless or non-physical, and genuine dimensional content concentrates at realization binding, where raw readings, motor angles, and sampling periods appear. Dimensional algebra is therefore mainly a Level-4 instrument, while nominal identity and port compatibility carry the earlier layers. This is also why the refusal of a global $\text{Real} \rightarrow \text{Brightness}$ coercion matters as much as it does: for most of the design surface it is the only thing standing between a well-formed model and a semantically wrong one.

Construction of a semantic value uses

$$\text{mk}_n : Q[d] \rightarrow \text{Sem } [n, d]$$

and may emit a validation obligation, for example $0 \leq b \leq 1$ for normalized brightness. The surface formula editor can insert this constructor during elaboration because the Mapping signature supplies the expected semantic codomain; the kernel itself contains no implicit nominal coercion.

4.3 Transduction: from readings to semantic quantities

The construction mk_n takes a physical quantity to a semantic one, but a sensor does not produce a physical quantity. It produces a device representation. BDL therefore uses a separate representation type $\text{Raw}[r]$, where r names the representation or sensor channel. Raw values do not participate in dimensional arithmetic merely because their storage happens to be numeric. The full path from device to design is

$$\begin{aligned} \text{Raw}[r] &\xrightarrow{\text{calib}} Q[d] \xrightarrow{\text{mk}_n} \text{Sem } [n, d] \\ &\xrightarrow{\text{lift}} \text{Signal } [d'] \text{Sem } [n, d]. \end{aligned}$$

Here the stages are respectively device representation, physical quantity, design property, and reactive design property.

and the output path is its mirror image, ending in a device write.

The first arrow deserves a name because it is where several distinct concerns actually live. A **calibration**

$$\text{calib} : \text{Raw}[r] \rightarrow Q[d]$$

is a pure function, authored in the same modes as a Mapping definition and carrying the same kinds of declared property. A realization module may expose an explicit decoder for the underlying numeric code of $\text{Raw}[r]$, but there is no general coercion $\text{Raw}[r] \rightarrow Q[0]$. This keeps representation arithmetic confined to the calibration boundary instead of granting raw ADC counts the algebra of physical dimensionless quantities. Calibration belongs to realization binding and does not appear on the intent canvas. Three properties of a calibration are worth declaring explicitly.

Direction. A raw reading need not increase with the physical quantity it measures. An ambient-light sensor reports larger values under stronger light, while many distance sensors report smaller values at greater range. The relation between the two directions is a fact about the device, and in conventional embedded code there is no location in which to record it, so it survives only as a sign buried in an expression. Declaring direction as part of the calibration gives that fact a place to live and lets the binding layer detect a reversed sensor.

Monotonicity, and hence invertibility. If a calibration is declared monotone, the tool can invert it. A designer who states a threshold in product terms, such as a distance below 10 cm, does not then compute the corresponding raw value; the tool derives it. Inverse propagation of thresholds through a declared calibration is one of the more directly useful consequences of typing this layer at all.

Nonlinearity. Thermistors, logarithmic photodiodes, and time-of-flight estimates that depend on the speed of sound are not affine. Placing them in a named, testable artifact separates device characterization from product logic, so that swapping a sensor changes a calibration rather than a behavior.

4.4 Value and reactive types

Core value types are

$$\begin{aligned} \tau ::= & \text{Unit} \mid \text{Bool} \mid \text{Int} \mid \text{Real} \mid \text{Raw}[r] \mid Q[d] \\ & \mid \text{Sem } [n, d] \mid \text{Enum } K \mid \tau_1 \times \tau_2 \\ & \mid \{\ell_i : \tau_i\}_{i \in I} \mid \text{Option } \tau. \end{aligned}$$

Reactive types are

$$\rho ::= \text{Signal } [d] \tau \mid \text{Event } [d] \tau.$$

Effect capabilities are tracked separately by an effect row ε , not embedded in ordinary value types.

4.5 Pure expressions

Pure expressions contain variables, constants, records, projections, conditionals, and registered pure operators:

$$e ::= x \mid c \mid p(e_1, \dots, e_n) \mid \text{ifethene}_1 \text{elsee}_2 \mid \{\ell_i = e_i\} \mid e.\ell.$$

They contain no assignment, I/O, general recursion, implicit clock reads, or arbitrary mutable state. This restriction is deliberate. Mapping formulas should be amenable to normalization, dimensional checking, range analysis, symbolic simplification, and deterministic pointwise evaluation.

The typing judgment is

$$\Delta, \Gamma \vdash e : \tau \triangleright \Phi,$$

where Δ is the device and primitive environment, Γ is the value environment, and Φ is a set of non-core validation obligations. Core typing does not depend on discharging Φ .

Representative rules include

$$x : \tau \in \Gamma \Rightarrow \Delta, \Gamma \vdash x : \tau.$$

and

$$(\Gamma \vdash e_1 : Q[d]) \wedge (\Gamma \vdash e_2 : Q[d]) \Rightarrow \Gamma \vdash e_1 + e_2 : Q[d].$$

For multiplication,

$$(\Gamma \vdash e_1 : Q[d_1]) \wedge (\Gamma \vdash e_2 : Q[d_2]) \Rightarrow \Gamma \vdash e_1 e_2 : Q[d_1 + d_2].$$

A trigonometric primitive may have type

$$\sin : Q[\text{Angle}] \rightarrow Q[0].$$

Whether angle is treated as its own design dimension or as dimensionless at the lowest numerical layer is a language-policy choice. BDL recommends preserving `Angle` in the design type system because doing so prevents a class of semantically meaningless connections.

4.6 Reactive nodes

The kernel graph contains a small set of node forms:

$$\begin{aligned} N ::= & \text{source} \mid \text{map } (f) \mid \text{emap } (f) \mid \text{filter } (p) \\ & \mid \text{rise} \mid \text{fall} \mid \text{delay } (v_0) \mid \text{temporal } (T) \\ & \mid \text{resample } (R) \mid \text{action } (op) \mid \text{scope } (H). \end{aligned}$$

A pure mapping rule is

$$\begin{aligned} & (\forall i. s_i : \text{Signal } [d]\tau_i) \wedge (x_1 : \tau_1, \dots, x_n : \tau_n \vdash f : \tau_o) \\ & \Rightarrow \text{map } (f)(s_1, \dots, s_n) : \text{Signal } [d]\tau_o. \end{aligned}$$

Its denotation is pointwise:

$$\begin{aligned} & \llbracket \text{map } (f)(s_1, \dots, s_n) \rrbracket(i) \\ & = \llbracket f \rrbracket(\llbracket s_1 \rrbracket(i), \dots, \llbracket s_n \rrbracket(i)). \end{aligned}$$

This is the formal basis of the surface-level `Tilt -> Brightness Mapping Block`.

4.7 Event extraction

For a Boolean signal, `rise` has type

$$\text{rise} : \text{Signal } [d] \text{ Bool} \rightarrow \text{Event } [d] \text{ Unit}.$$

For $i > 0$,

$$\text{rise } (s)(i) = \text{Some } (())$$

iff $\neg s(i-1) \wedge s(i)$. The initial tick follows an explicit initialization policy. This primitive replaces the common procedural pattern of maintaining a previous Boolean value manually.

4.8 Explicit synchronization

Synchronization is the only operator that changes the domain index of a value. It is never inserted implicitly, and it is a visible node in the design.

4.8.1 Signals: zero-order hold

If

$$s : \text{Signal } [d_1]\tau,$$

then using it in d_2 requires

$$\text{hold}_{d_1 \Rightarrow d_2}(v_0, s) : \text{Signal } [d_2]\tau.$$

Once both domains are bound, let the target tick occur at physical time $t = \kappa_{d_2}(j)$. BDL defines synchronization as a **causal boundary**: the target may observe only source values committed strictly before t . Let

$$i^* = \max\{i \mid \kappa_{d_1}(i) < t\}.$$

If such an i^* exists, the result is $s(i^*)$; otherwise it is the explicit initial value v_0 . A source tick that happens at the same physical timestamp as the target tick is committed only after the current global step and therefore becomes visible at a later target tick. This strict inequality prevents a cross-domain zero-time dependency from reintroducing an algebraic cycle through synchronization. The initial value is required, not defaulted, because it states the product's behavior before the source has ever reported. In a safety context this is a design decision, not an implementation detail.

4.8.2 Events: an explicit policy is required

Holding is meaningful for signals because a signal has a value at every tick, so reusing the previous one is well defined. An event has no value at most ticks. Transporting events from d_1 to d_2 must therefore answer a question that has no default answer: if three occurrences fall inside one target tick, which of them are observed, and when.

A cross-domain event node carries an explicit policy

$$\pi ::= \text{buffer} \mid \text{coalesce } (\mu) \mid \text{latest} \mid \text{drop},$$

with

$$\text{sync}_{d_1 \Rightarrow d_2}^\pi(e) : \text{Event } [d_2]\tau.$$

The synchronizer is stateful. Occurrences produced at source times strictly earlier than a target tick are first accumulated in its pending state; occurrences at the same physical timestamp as the target tick are committed afterwards, under the same causal-boundary convention as signal `hold`. `buffer` releases at most one queued occurrence per target tick and preserves order and count. `latest` releases the most recent pending occurrence and discards older ones. `drop` releases the earliest pending occurrence and discards the rest. `coalesce` is not defined for an arbitrary payload type: it requires a deterministic explicit merge function $\mu : \tau^+ \rightarrow \tau$ (for `Unit`, the canonical merge simply returns `()`).

The mathematical kernel may model the pending buffer as unbounded. A concrete realization with bounded storage emits a queue-capacity and overflow obligation; it does not silently change the event policy. The choice is observable at the product level: “the button was pressed three times” and “the button was pressed” are different design statements. A sample-and-hold semantics is never silently reused for events.

5 State and Temporal Semantics

5.1 Temporal operators are stateful transducers

Surface temporal phrases elaborate to finite-state transducers. For example, `p for d` denotes a Boolean signal that becomes true only after predicate p has remained continuously true for duration d . The kernel implementation maintains elapsed time internally, but this state is not user-visible unless inspected.

A temporal primitive has a transition form

$$T : (\sigma, I_i, \Delta t_i) \mapsto (\sigma', O_i),$$

where σ is private local state. The primitive is deterministic and total for well-typed inputs.

5.2 StateHandler activation

A StateHandler H has an activation stream $a_H : \text{Signal } [d] \text{ Bool}$, a local graph G_H , local machine state Σ_H , child handlers, and an action policy. A condition-scoped handler simply uses

$$a_H(i) = p(i).$$

An event-latched handler with entry event en_H and exit event ex_H uses an explicit deterministic policy. Under the default **exit-wins** policy,

$$a_H(0) = \text{has}(en_H(0)) \wedge \neg \text{has}(ex_H(0)),$$

and

$$a_H(i+1) = \neg \text{has}(ex_H(i+1)) \wedge (a_H(i) \vee \text{has}(en_H(i+1))).$$

If same-tick exit and re-entry is desired, the designer must choose a separate **reenter** policy. The runtime does not infer it.

If H_c is a child of H_p , effective activation is

$$a_{H_c}^*(i) = a_{H_p}^*(i) \wedge a_{H_c}(i).$$

5.3 Activation stratification

A handler's activation condition is authored in its **parent scope**. It may depend on sampled or synchronized inputs, pure mappings available in the parent, and committed pre-tick temporal state, but it may not depend on a same-tick value whose evaluation is guarded by that handler or by one of its descendants. Equivalently, if the elaborated dependency graph contains an edge from the body of H back into the computation of a_H , the model is rejected unless the cycle crosses an explicit temporal boundary. This stratification prevents a scope from needing to be active in order to compute whether it is active.

5.4 Reset-on-entry local state

The default local-state policy is **reset on entry**. A temporal operator placed inside `HeId` should not continue counting while the product is not held. All values inside the handler retain the enclosing reactive type $\text{Signal } [d] \tau$ or $\text{Event } [d] \tau$; activation does not manufacture a new nominal clock domain. Define the active-tick set

$$\text{ActiveTicks}(H) = \{i \in \mathbb{N} \mid a_H^*(i) = \text{true}\}.$$

This set is an **operational subclock view**, not a domain index and never appears in a reactive type. Local temporal state advances only on ticks in $\text{ActiveTicks}(H)$ and is reset at each inactive-to-active boundary. On inactive ticks the handler emits no local action requests and its resettable local state does not advance. Persistent history is available only through an explicit **persistent** memory declaration. This makes persistence a visible design decision rather than stale runtime residue.

5.5 Controlled history primitives

BDL exposes a small vocabulary of semantic history operators:

`previous, count, since, average, hold, latch`.

For example, “picked up more than three times within ten minutes” can be represented as

$$\text{count}(\text{PickedUp}, 10 \text{ min}) > 3$$

without exposing an integer counter, reset branch, or timer variable to the designer.

6 Action Requests and Scoped Policies

6.1 Effect signatures

The device environment Δ assigns each request operation a parameter type,

$$\Delta(op) = P_{op}.$$

The first kernel is deliberately **request-only**: issuing an operation does not synchronously return a value. A well-typed runtime request is therefore a dependent pair

$$\text{Request}_\Delta = \sum_{op \in \text{dom}(\Delta)} P_{op}.$$

For example, the surface capability declarations may be written

```
request set_brightness(Brightness)
request move_to(MotorAngle)
request play_sound(SoundId)
```

An action node consumes a signal or event carrying the corresponding operation parameter and emits a request on the same logical clock. Acknowledgement, failure, and asynchronous completion are represented neither by a phantom synchronous result type nor by `Unit`; they are a separate extension discussed under formal limits. The behavior judgment tracks a row of possible operations:

$$\Gamma \vdash b : \tau! \varepsilon.$$

6.2 Policy interpretation

A StateHandler policy may allow, deny, transform, cap, prioritize, or arbitrate requests. Statically, a policy is modeled as an effect-row transformation

$$H : \varepsilon_{in} \Rightarrow \varepsilon_{out}.$$

Dynamically, it is a total deterministic transformer on the finite request multiset produced at one tick. Nested policies are applied inside-out:

$$\text{Resolve}_{H_p \circ H_c}(R) = \text{Resolve}_{H_p}(\text{Resolve}_{H_c}(R)).$$

This ordering permits local context to modify behavior while ensuring that outer safety policies still see every forwarded request.

The first BDL kernel deliberately stops here. It does not expose general resumable continuations and does not claim that `StateHandler` is a general algebraic-effect handler. The semantic relationship is narrower: both separate a request from its interpretation, and a future calculus may adopt scoped algebraic effects if it becomes necessary to model richer context-sensitive interactions [10].

6.3 Actuator conflicts

If two potentially simultaneous requests target the same exclusive actuator channel, the model is accepted only if at least one of the following holds:

1. static analysis proves the requests mutually exclusive;
2. an explicit deterministic arbitration policy is present; or
3. the device operation is declared commutative and has a specified composition law.

Otherwise the checker reports `UnresolvedEffectConflict`. There is no “last wire wins” rule.

7 Static Semantics and Typechecking

7.1 Bidirectional expression checking

BDL uses bidirectional typing because Mapping Blocks almost always supply an expected codomain. Algorithmic judgments are

$$\Delta, \Gamma \vdash e \Rightarrow \tau \dashv C$$

for synthesis and

$$\Delta, \Gamma \vdash e \Leftarrow \tau \dashv C$$

for checking. Here C contains decidable structural constraints.

A typed hole has only a checking rule:

$$\Delta, \Gamma \vdash ?_m \Leftarrow \tau \dashv \{?_m : \tau\}.$$

and no unconstrained synthesis rule. A hole therefore cannot invent a universal type. It is always constrained by a signature, port, or explicit annotation.

7.2 Signature-first Mapping checking

Suppose the designer has declared

$$m : (A_1, \dots, A_n) \rightarrow B.$$

The editor constructs

$$\Gamma_m = x_1 : A_1, \dots, x_n : A_n.$$

If the mapping definition is a hole, the checker records

$$?_m : (A_1, \dots, A_n) \rightarrow B.$$

If a formula e is supplied, the preferred rule is expected-type checking:

$$\Delta, \Gamma_m \vdash e \Leftarrow B \dashv C.$$

For nominal semantic outputs, the surface elaborator may insert the explicit kernel constructor implied by B and emit any associated range obligation. For example, a scalar formula attached to a Brightness output elaborates to `mk_Brightness(e)` only because the surrounding signature already states that the relationship produces brightness. The core calculus does not permit a global `Real` $\rightarrow$ `Brightness` coercion.

This is an important usability choice. The user does not repeatedly write nominal constructors inside formulas, yet the formal model remains explicit after elaboration.

7.3 Constraint domains

Core constraints are separated into four decidable families:

$$C = C_{type} \cup C_{dim} \cup C_{domain} \cup C_{row}.$$

C_{type} contains structural and nominal equalities; C_{dim} contains integer-vector dimension equations; C_{domain} contains domain-name equalities, decided syntactically rather than by unification; C_{row} contains effect-row equalities and inclusions. Refinement predicates, physical ranges, deadlines, bus bandwidth, and other engineering properties are *not* placed in this unification problem.

This separation prevents an SMT solver from becoming an accidental oracle for type soundness.

7.4 Port compatibility

An ordinary wire is valid only when its source and destination reactive types match after representation-preserving unit normalization, and when their domain indices are syntactically equal. The following are never inserted implicitly:

- `Signal` $\rightarrow$ `Event`;
- `Event` $\rightarrow$ `Signal`;
- a direct cross-domain signal or event wire when $d_1 \neq d_2$;
- a dimension-changing conversion;
- a stateful temporal conversion;
- actuator arbitration.

These are semantic operations and therefore must be visible design artifacts or properties.

7.5 Causality

Let G be the elaborated reactive graph. Construct the instantaneous dependency graph G_0 by cutting edges that cross an explicit one-tick or temporal state boundary. BDL requires G_0 to be acyclic. Equivalently, every non-trivial strongly connected component must contain a declared delay boundary before the graph is accepted.

This excludes instantaneous algebraic loops from the executable kernel and gives a topological evaluation order per tick. It follows the synchronous-language tradition of making causality a static property [11].

7.6 Model acceptance levels

The checker distinguishes three claims:

- **Well-typed partial.** Every unresolved hole has a coherent expected type and the surrounding graph is structurally type-correct.
- **Executable.** No runtime-required hole remains; domain rates, causality, effect policies, and device capabilities are resolved.
- **Verified(P).** The model is executable and the selected validation obligations P have been discharged.

A green check mark should never collapse these claims into one. “The model can run” is weaker than “the model satisfies the chosen physical and safety properties.”

8 Validation Obligations

Core typing answers questions such as “is this value a brightness rather than a temperature?”, “do these signals share a domain?”, and “can this action be issued in this scope?” It should not attempt to decide every engineering property.

The elaborator therefore produces a set Φ of proof or analysis obligations. Typical obligations include

- semantic range preservation, e.g. $0 \leq L \leq 1$;
- monotonicity of a Mapping;
- maximum response latency;
- actuator update-rate limits;
- bus utilization and scheduling;
- motor velocity or acceleration feasibility;
- mutually exclusive mechanical configurations;
- safety invariants.

An obligation may be discharged by interval analysis, symbolic algebra, linear or nonlinear solving, model checking, simulation evidence, randomized property testing, an explicit runtime guard, or an unverified declaration by the author of a supplied component. The mechanism is recorded with the result, and the mechanisms are not equivalent. A range obligation established by interval analysis over a normalized expression, one established by a million randomized cases, and one asserted by a component vendor are three different epistemic situations.

Verified (P) must therefore report how P was obtained. A model whose safety property rests on a declared worst-case execution time is not in the same state as one whose property was proved, and presenting both with the same indication would reintroduce exactly the false confidence the acceptance levels are meant to prevent.

9 Tick Semantics

9.1 Machine configuration

Let g index the global schedule κ_G , and let

$$F_g = \text{Fire } (g)$$

be the set of domains whose local clocks tick at physical time $\kappa_G(g)$. A runtime configuration is

$$\mathcal{M}_g = \langle I_g, \Sigma_g, A_g \rangle,$$

where I_g contains sampled inputs and synchronized event deliveries for the firing domains, Σ_g contains temporal state, synchronizer state, persistent memory, and the last committed source values, and A_g is the set of active StateHandlers.

One global step proceeds in five phases:

1. **Sample.** For each $d \in F_g$, acquire external inputs for that domain and expose synchronizer outputs computed only from source data committed at physical times strictly earlier than $\kappa_G(g)$.
2. **Activate.** Evaluate stratified StateHandler activation predicates in the firing domains and apply reset-on-entry boundaries.
3. **Evaluate.** Evaluate the instantaneous DAGs of the firing domains in topological order using the pre-step state Σ_g . Domains that do not fire do not advance their local temporal state.
4. **Resolve.** Collect action requests produced at this global instant and apply nested deterministic policies and arbitration.
5. **Commit.** Commit temporal-state updates, newly sampled source values, and event-queue updates to obtain Σ_{g+1} , then hand resolved requests to the realization layer.

The ordering of **Sample** before **Commit** is semantically important. If a source and target domain tick at the same physical timestamp, the target observes the source's previously committed value or pending events, never the source's just-computed value from the same global step. This gives every explicit cross-domain synchronizer a causal boundary and prevents simultaneous cross-domain dependencies from forming an instantaneous cycle. State updates are otherwise committed only after instantaneous evaluation, preventing accidental order dependence among nodes that are conceptually simultaneous.

9.2 Determinism theorem

Let G be an executable kernel graph satisfying:

1. all primitive expression and temporal operators, and all supplied computation blocks, are deterministic and total on well-typed inputs;
2. G_0 is acyclic;
3. every domain index is syntactically resolved and every cross-domain transition is an explicit synchronization node with a fixed deterministic policy;
4. all StateHandler activation policies are deterministic;
5. every action policy is total and deterministic; and
6. all exclusive actuator conflicts are statically resolved.

Then for any global step g , any well-typed machine state $\mathcal{M}_g$, and well-typed external inputs for the domains in F_g , there exists a unique next configuration $\mathcal{M}_{g+1}$ and a unique resolved action multiset R_g .

Proof sketch. The global schedule uniquely determines the firing-domain set F_g . Synchronizer outputs are functions only of committed pre-step state, so simultaneous source and target ticks cannot introduce a scheduling choice. Stratified activation is a deterministic function of sampled inputs and prior state. Acyclicity of G_0 gives a unique topological evaluation of every firing domain's pure and reactive instantaneous nodes; deterministic primitives therefore produce unique node values. Temporal nodes compute unique pending state updates from the pre-step state. Total deterministic policy composition yields a unique resolved request set. The commit phase

then yields a unique next state. No phase depends on arbitrary node scheduling.

9.3 Type preservation across a tick

Let $\text{WT}(\mathcal{M}, G)$ mean that all runtime cells, active-handler states, synchronizer cells, and bound device values conform to the types assigned by the elaborated graph. If

$$\text{StaticAccept}(G), \quad \text{WT}(\mathcal{M}_g, G), \quad \text{WTInput}(I_g, G),$$

and

$$\text{Step}_G(\mathcal{M}_g, I_g) = (\mathcal{M}_{g+1}, R_g),$$

then

$$\text{WT}(\mathcal{M}_{g+1}, G)$$

and every request (op, p) in R_g satisfies $p : P_{op}$ for the corresponding declaration $\Delta(op) = P_{op}$ and names an operation present in the inferred effect row.

The proof follows by induction over the topological node order plus preservation lemmas for temporal cells and policy transformations. This paper states the proof obligations precisely but does **not** claim a mechanized proof. A mechanization would be a required step before presenting the calculus as a completed PL metatheory rather than a language-design proposal.

10 Elaboration Pipeline

The surface editor and kernel are connected by an elaboration function

$$\mathcal{E} : \text{Surface} \rightarrow \text{KernelPartial} + \text{Diagnostics}.$$

A practical implementation can use the following passes.

10.1 Name and signature resolution

Resolve semantic properties, Mapping signatures, StateHandler scopes, abstract device capabilities, units, and imported components. At this stage an unresolved `Tilt` $\rightarrow$ `Brightness` mapping already has a stable identity and type.

10.2 Formula elaboration

For each Mapping with a definition, construct the local environment from the input signature and check the definition against the declared codomain. Surface syntax such as a scalar expression for a nominal output is elaborated into the corresponding explicit kernel constructor. Symbolic simplification may run after typing, never as a substitute for it.

10.3 Reactive lifting

A pure signature is lifted over signal or event ports according to an explicit lifting rule. Pointwise signal mapping, event payload mapping, filtering, and event extraction are distinct operators. The elaborator must not silently turn one category into another.

10.4 Domain propagation

Because domains are declared rather than inferred, this pass is a single bottom-up traversal rather than a constraint-solving problem. Every source node states its domain. Every other node derives one:

- `map`, `emap`, `filter`, `rise`, `fall`, `delay`, and `temporal` with a single reactive input inherit that input's domain;
- a node with several reactive inputs requires all of them to carry the same domain, and reports a diagnostic otherwise;
- `hold` and `sync` are the only nodes that change a domain, and both are explicit;

- `scope(H)` preserves the enclosing domain d ; it adds an activation mask and reset policy but does not change the reactive type index.

A node's domain is therefore fixed when the traversal reaches it, with no backtracking, no constraint set, and no provenance reconstruction. The diagnostic for a domain mismatch is located at the multi-input node itself, which is the wire the designer drew, and it is phrased in the names the designer chose:

```
critical is updated in ambient; heating control is in
interaction. Hold the last temperature verdict, or move heating
control to ambient?
```

A useful diagnostic states *what behavioral consequence each option has*, and both options above are legitimate designs rather than error recoveries.

10.5 State and temporal lowering

Surface phrases such as `for 300 ms` and `while Held lower` to typed temporal transducers and `StateHandler` activation streams. Local temporal state is annotated with reset behavior at scope boundaries.

10.6 Effect-row accumulation and conflict analysis

Collect operations reachable in each scope, apply policy-row transformations, then analyze potentially simultaneous writers for each exclusive actuator channel. When mutual exclusion cannot be proven, the tool asks for an arbitration policy instead of guessing.

10.7 Normalization and erasure

After typing, mapping definitions are normalized. Normalization by evaluation is convenient here because the pure fragment is deliberately free of assignment, general recursion, and implicit clock reads, so strong normalization is inexpensive, and because host operators can be added to the semantic domain individually, without extending the syntax or the typing rules. It should be noted that the kernel is not dependently typed and does not require normalization to decide type equality; this pass is an analysis and code-generation instrument rather than part of typechecking.

It pays for itself in three places. Nominal wrappers introduced by elaboration are eliminated: with $\text{mk}_n(\text{rep}(x)) \rightsquigarrow x$ and $\text{rep}(\text{mk}_n(e)) \rightsquigarrow e$, the semantic layer disappears from the residual term. Interval analysis becomes sharper, because interval arithmetic is sensitive to syntactic form and a normalized expression yields tighter bounds than the form the designer wrote. After realization binding, constants introduced by the binding are folded, which is what makes a generated implementation compact enough for a microcontroller.

The same argument justifies erasure at code generation. Because checking is complete before erasure, semantic names, dimensions, and domain indices carry no computational content and may be removed without changing behavior: the type layer bears the whole semantic burden and leaves no runtime residue. Erasure is nonetheless applied selectively. At boundaries the checker cannot see through, namely supplied computation blocks, device bindings, and the public interface of the generated code, nominal wrappers are retained so that the host compiler continues to check what BDL cannot, and so that the generated code remains legible to the engineer who receives it.

One decision must be made explicitly rather than inherited. Floating-point addition is not associative, so an equational theory over the reals and an equational theory over machine floats do not agree. BDL normalizes symbolically over the reals and records the resulting numerical deviation as an obligation, while the evaluation path performs constant folding only. A range result therefore states which

of the two theories it was established in. Without this separation, normalization would silently alter the property being verified.

10.8 Validation generation

Finally emit range, timing, capacity, and safety obligations. These do not determine whether the graph is type-correct; they determine whether stronger claims such as `Verified({range, latency})` are justified.

11 Interaction Design

The formal language is useful only if its surface prevents the very engineering burden it is intended to remove. The editor should therefore be layered around the sequence in which product intent normally becomes precise.

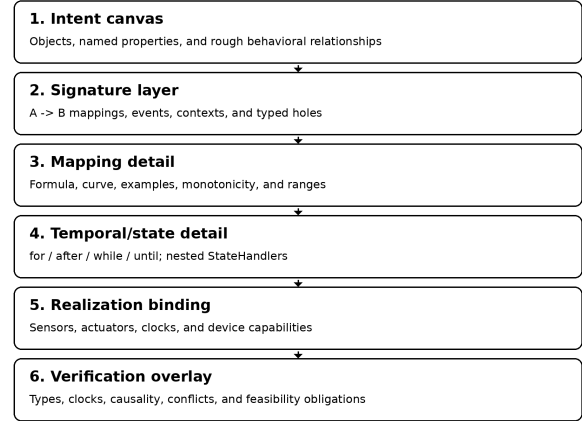


Figure 2: Recommended progressive authoring layers. Each layer adds semantic commitment without invalidating the higher-level design artifact.

11.1 Layer 1: Intent canvas

The first view contains product objects, named properties, events, contexts, and relationships. The designer can create `Tilt -> Brightness` without choosing a formula and can create `CupPickedUp` without choosing a sensor. The canvas should be sparse enough to remain a product-behavior diagram rather than a circuit diagram.

A Mapping Block should visually emphasize its signature. A useful shape is:

```
+-----+
| Tilt  ---- ?f ----> Brightness |
+-----+
```

The unresolved `?f` is not an error badge. It communicates that the relationship is semantically declared but not yet defined.

11.2 Layer 2: Signature and property inspector

Opening the block reveals domain and codomain documentation, units, ranges, clock expectations, and optional semantic properties. At this stage the user may state “monotonically increasing” or add input-output examples without writing a formula.

This layer is important because it allows a middle ground between a vague arrow and a complete mathematical implementation. A relationship can become progressively more constrained while remaining open.

11.3 Layer 3: Mapping detail

The designer can choose one of several equivalent authoring modes:

- formula editor;
- piecewise curve editor;

- direct-manipulation graph;
- table of examples with fitting;
- reference to a reusable component.

All modes elaborate to the same typed mapping boundary. The formula is an *attachment to the relationship*. It never becomes another box on the main flow path.

For expert users, the formula editor can display inferred units and the expected type continuously. For novice users, unit errors should be phrased in product terms, e.g. “This expression produces angular velocity, but this block promises brightness,” rather than exposing a unification trace.

11.4 Layer 4: Temporal and StateHandler detail

Temporal modifiers appear as annotations or scope boundaries rather than ordinary nodes whenever possible. For example, a block may carry a `for 300 ms` badge, and a `Held StateHandler` can visually contain the mappings that apply while the cup is held.

Entering a `StateHandler` should feel similar to entering a component or assembly: the designer edits the behavior valid in that context. The UI can offer an optional state-tree navigator, but the default canvas need not resemble a conventional statechart.

11.5 Layer 5: Realization binding

Only when the designer chooses to bind the concept to hardware does the tool expose concrete sensors, actuators, sample rates, bus interfaces, and capability limits. Tilt may be bound to an IMU-derived estimate; Brightness may be bound to a PWM light channel.

This is where abstract clocks become concrete and where previously dormant feasibility obligations become actionable. The conceptual design remains intact if the user later swaps an IMU or actuator, because the behavior model depends on semantic ports rather than device APIs.

11.6 Layer 6: Verification overlay

Verification should be displayed as an overlay on the design, not as a separate engineering universe. The UI should distinguish at least:

- type-correct;
- incomplete but type-consistent;
- executable;
- obligation pending;
- obligation disproved;
- verified for a named property set.

Diagnostics should attach to the relevant design relationship. If a 1 Hz sensor cannot support a 300 ms pickup detector, the message belongs on the detector or binding, not in a compiler console full of clock-variable names.

12 Worked Example: Smart Cup

Consider a cup with a contact sensor, an orientation estimate, a light, and a heater. The design intention is:

1. while the cup is held, light brightness increases with tilt;
2. a high temperature warns the user;
3. a critical temperature disables heating regardless of the current interaction context.

The initial canvas can contain only the following declarations:

```
domain interaction
domain ambient
```

```
Held : Context
```

```
Contact      : Signal[interaction] Contact
Tilt          : Signal[interaction] Tilt
Temperature   : Signal[ambient] Temperature
```

```
?pickup      : Contact -> Event<PickedUp>
?f           : Tilt  -> Brightness
WarnHot      : Temperature -> Event<Warning>
request SetBrightness(Brightness)
request DisableHeating(Unit)
```

At this stage `?pickup` and `?f` are holes, but the graph is already a well-typed partial design. The designer can place `?f` inside `Held` and connect its output to `SetBrightness` without having chosen an IMU, sampling rate, or formula.

The two domain declarations record a product judgment, not an engineering one: contact and orientation are quantities that move with the interaction, temperature is a quantity that moves with the environment. No rate has been chosen, and none is needed to make that statement.

Later, the mapping is refined:

$$f(\theta) = \text{mk}_{\text{Brightness}}\left(\text{clamp}\left(0.2 + 0.8 \frac{\text{rep}(\theta)}{60^\circ}, 0, 1\right)\right).$$

The surface editor may display only the scalar formula and infer the nominal constructor from the signature. The kernel records the constructor and emits the brightness range obligation.

Pickup detection may then be specified as “contact false for 300 ms,” which elaborates to a temporal transducer and rising-edge event. When the concrete contact sensor is later bound at 20 Hz, the timing is representable. If the designer instead binds a 1 Hz sampled source, the core graph remains type-correct, but a feasibility obligation for the 300 ms detector fails. This distinction is intentional: *type correctness and physical adequacy are different claims*.

The safety rule is represented as an outer policy that suppresses heater-enabling actions while critical temperature holds. Because outer policies resolve after inner interaction contexts, the safety rule does not have to be duplicated into every state. This avoids a common source of state explosion and makes the safety intent visibly global.

That rule also crosses a domain boundary, and this is where the treatment of time becomes visible. The critical-temperature predicate is computed in `ambient`; the heating action it suppresses is issued in `interaction`. The checker rejects the direct connection at Level 3, while the designer still holds every piece of information needed to resolve it and still lacks none of it, and asks which behavior is intended: hold the last temperature verdict, or move heating control into `ambient` and accept a slower response. Choosing the first inserts

```
criticalHeld =
  hold[ambient => interaction](false, critical)
```

as a visible node. Its initial value states what the product does before the first temperature reading has ever arrived, which for a safety interlock is a decision that should be written down rather than defaulted.

At realization binding the domains acquire rates, say `interaction` at 50 Hz and `ambient` at 1 Hz, and the devices acquire theirs: an IMU capable of 100 Hz, a contact sensor at 20 Hz, a temperature sensor at 1 Hz. The rate disagreement inside `interaction` is not a type error, because domain agreement was settled at Level 3; it is a set of obligations. Downsampling the IMU requires a stated policy. Under-sampled contact readings are held, with a bounded staleness of one domain tick. The 300 ms detector spans fifteen ticks and is feasible. Had the contact sensor been bound at 1 Hz, that last obligation would fail while the graph remained type-correct, which is exactly the distinction the acceptance levels are meant to preserve.

12.1 A second example: a five-axis mechanical printer

The Smart Cup is small enough to be read at a glance and correspondingly weak as evidence. A second and larger case is a mechanical typewriter driven as a printer, built by one of the authors: a client application drives an embedded controller which coordinates five motors over a CAN bus.

Its architecture was authored in a manner close to the one this paper advocates, though not with these tools. The front-end and back-end boundary was fixed first, then the protocol across it, and only afterwards the bus-level detail. The protocol carries an atomic motion block, an acknowledgement, and a motion-status report. Two observations follow.

First, this ordering is interface-first rather than mapping-first. Component boundaries and message formats were settled early; the individual quantitative relationships were not, and were largely written at the moment they were implemented, because for an author who already knows the transfer function there is no interval between declaring it and defining it. This is examined further in the evaluation plan.

Second, and more consequentially for the calculus, the protocol exposes a genuine limitation. A BDL `SetBrightness` request is intentionally fire-and-forget in the first kernel, whereas a motion command in the printer protocol is acknowledged, may fail, and reports asynchronous status. Real actuators stall, exceed travel, and time out. The example is used here only to motivate the need for asynchronous outcomes; quantitative claims about arbitration or timing are deferred until measured traces and protocol facts are incorporated, rather than being inferred from the existence of the implementation.

13 Usability Analysis and Evaluation Plan

A language for designers must be evaluated on more than compilation success. The central hypothesis is that signature-first, declarative behavior modeling reduces the need to translate design intent into implementation structures while preserving enough precision for early validation.

13.1 Expected cognitive advantages

Lower viscosity. Changing a transfer function edits one Mapping definition rather than a procedural chain of read/compute/store/write nodes.

Reduced hidden dependencies. Semantic signatures expose what a relationship consumes and produces, while the kernel rejects incompatible connections before device code exists.

Reduced premature commitment. Typed holes allow a designer to commit to a relationship without committing to its implementation, sensor, or exact parameters.

This last item requires a qualification that the rest do not. Signature-first authoring is not claimed to be the spontaneous habit of every designer. One of the authors, working on the hardware project described above, adopted it when the complexity of a subsystem exceeded what could be held in view at once, and did not adopt it on simpler tasks, where a signature and its definition were written in a single motion. Expertise does not remove the strategy; it raises the threshold at which it appears.

The hypothesis is therefore narrower and more testable than a claim about how designers think. It is that the strategy remains effective for less experienced designers, but that conventional tools provide no legal position for an undefined relationship, so it cannot be adopted even when it would help. A hole is that position. On this reading the contribution is not that the tool matches an existing habit, nor that it should train a new one, but that pausing between declaring a relationship and defining it should cost nothing. Whether inexpe-

rienced designers do take up the strategy once it is available, and whether doing so improves their designs, is an empirical question and is included in the study below.

Better role expressiveness. StateHandler, Mapping, and temporal modifiers correspond to product-design concepts rather than generic program-control constructs.

Improved error locality. A mismatch is attached to a product relationship or binding rather than surfacing later as an embedded runtime fault.

13.2 Usability risks

The proposal also creates new risks.

Type proliferation. If every semantic distinction creates a visible type, the editor may become bureaucratic. The system needs reusable type libraries, inference from connected ports, and sensible defaults.

Formula anxiety. Not every designer wants to write equations. Formula authoring must therefore coexist with curves, examples, and direct manipulation.

False confidence. A formally typed diagram can look “verified” even when no physical feasibility property has been checked. The UI must preserve the acceptance-level distinction. The risk is sharpest for obligations discharged by declaration rather than by analysis: a model whose latency guarantee rests on a worst-case execution time asserted by the author of a supplied component is verified in a materially weaker sense than one whose guarantee was computed, and the two must not be displayed alike.

Hidden elaboration. Hiding too much implementation can make runtime behavior mysterious. The tool needs an inspectable explanation view showing the generated reactive structure and state when requested.

Semantic ambiguity in names. Tilt is meaningful to humans but its precise definition may vary. A semantic type therefore needs documentation and a binding contract, not merely a label.

13.3 Comparative study

A first controlled study should compare BDL against at least two baselines: a statechart-based prototyping environment and a node-based or Arduino-style implementation workflow. Participants should be industrial-design students and practitioners with limited professional software-engineering experience.

Tasks should include: (1) specifying a sensor-to-actuator mapping; (2) adding temporal qualification such as “for 300 ms”; (3) adding an orthogonal safety override; (4) replacing a sensor with a different sample rate; (5) relating a slowly updated quantity to a fast interaction, which forces a cross-domain decision; and (6) modifying a mapping late in the task.

Primary outcomes should not be limited to task time or SUS. More important measures are:

- semantic errors in the final behavior;
- number of implementation-only concepts participants must manipulate;
- time to detect an impossible or conflicting behavior;
- fidelity between verbal design intent and executable model;
- number and diversity of behavior alternatives explored;
- quality of handoff to an engineer who did not observe the authoring session;
- subjective confidence calibrated against actual correctness;
- whether, and at what level of task complexity, participants declare a relationship before defining it when the tool permits both.

The last measure tests the hypothesis stated earlier rather than assuming it. Because signature-first authoring may be a strategy that appears only above a complexity threshold, the tasks should vary in scale, and the default state of a newly created Mapping Block is itself a manipulable factor: a block that opens onto an empty formula editor and one that opens onto a signature with an explicitly legal undefined body invite different first actions. A small comparison of these two defaults is considerably cheaper than the full study and can be run during Stage 1.

A particularly important test is whether BDL increases *design agency*: can participants directly specify and revise behaviors that they would otherwise delegate or avoid because the implementation representation is too costly?

13.4 Field study

A controlled study cannot establish whether the representation fits real design practice. A second phase should embed the tool in a semester-long product-design studio or an industry project. The study should observe where typed holes persist, which semantic types designers invent, where they request escape hatches, and how often engineers reinterpret or replace BDL artifacts during implementation.

This field evidence is necessary before claiming that the language is “native” to industrial design rather than merely pleasant to its authors.

14 Relationship to Prior Work

14.1 Physical prototyping and design tools

Phidgets reduced the implementation cost of physical interaction by presenting hardware components through a uniform software abstraction [1]. d.tools went further by integrating physical prototyping, statechart-based behavior, testing, and analysis for designers [2]. BDL shares the goal of keeping designers focused on behavior rather than low-level implementation, but shifts the primary representation from statechart execution structure to typed semantic relationships with progressively supplied definitions.

Exemplar addressed the same authoring cost from the opposite direction, letting designers demonstrate sensor behavior and having the system infer the recognizer [12]. In BDL that technique is one of several ways to supply a definition, attached to a signature that already fixes the relationship’s semantic boundary.

The difference is not that prior systems could not represent behavior. They could. The claim is narrower: their dominant representation still asks designers to formulate substantial portions of behavior in an engineering-oriented operational structure. BDL investigates whether the *type signature* and *semantic relationship* can be the primary design artifact, with operational machinery elaborated underneath.

14.2 Dataflow and model-based engineering tools

Several mature systems already offer graphical behavior authoring to users who are not primarily programmers. LabVIEW established the graphical dataflow instrument-control paradigm; Simulink with Stateflow combines dataflow blocks with hierarchical state machines and dominates control-system practice; Modelica models physical systems through acausal equations with units and dimensions [13], [14], [15]. These systems are considerably more capable than what is proposed here, and BDL should not be read as competing with them on coverage.

The distinction is in the primary artifact and the intended author. In each of these systems the artifact is an executable model whose blocks denote computation, and the author is expected to hold an engineering model of the system. BDL’s artifact is a set of typed semantic relationships that need not yet compute anything, and

the author is expected to hold a product model. Whether this is a difference worth a new system, rather than an interface layer over an existing one, is exactly what the evaluation below is meant to determine.

It should also be stated plainly that the kernel is assembled from established components rather than invented. Dimensional typing follows the units-of-measure line begun by Kennedy [16]; clocks-as-types, causality as a static property, and synchronous tick semantics come from the synchronous dataflow tradition; signals and events come from functional reactive programming; typed holes come from work on structure editing. The claim to novelty concerns the representation and the interaction model, and specifically the decision to make nominal semantic relationships primary and to author temporal identity rather than infer it. It does not concern the calculus.

14.3 Statecharts and synchronous reactive languages

Statecharts provide hierarchy, concurrency, and communication for complex reactive systems [6]. Synchronous dataflow languages and their state-machine extensions provide precise clock and causality semantics suitable for safety-critical implementation [9], [11], and Zélus extends the same lineage to hybrid systems combining discrete and continuous dynamics [17], which is the direction in which the limitations noted above would have to be addressed.

BDL borrows these semantic strengths while refusing to make their operational concepts the default designer-facing notation. The refusal extends further than notation in one respect. Where those languages recover clocks by inference, BDL requires temporal identity to be declared, on the grounds set out in the discussion: inference relocates each temporal decision to realization binding, which in this workflow is the point at which the designer is least able to make it.

14.4 Functional reactive programming

FRP established behaviors/signals and events as compositional abstractions for time-varying computation [4]. FrTime demonstrated a dynamic dataflow embedding with formal semantics [5]. BDL uses a much more restricted reactive core because product-behavior tooling benefits from explicit clocks, bounded temporal state, and predictable causality.

14.5 Typed holes and structure editing

Hazelnut demonstrates that incomplete structured terms can remain statically meaningful under bidirectional typing [8]. BDL treats this idea as directly relevant to design. A typed hole represents an intentional open design decision. Unlike a conventional programming environment, the incompleteness may persist across significant portions of the design process and is part of the workflow rather than merely a temporary editing state.

14.6 Algebraic and scoped effects

Algebraic effects separate operation requests from handlers that interpret them [7]. Scoped-effect work explores operations whose meaning extends over structured regions [10]. BDL borrows the separation principle for actuator requests and StateHandler policies but currently defines a simpler deterministic request transformation semantics. A stronger effect calculus should be adopted only if concrete product-design use cases require resumable control or more general scoped operations.

14.7 Model-based systems engineering

SysML and MBSE address precise system structure, requirements, behavior, and verification at a broader engineering level [18]. BDL is not intended to replace them. Its research question is whether a substantially smaller, design-native behavioral notation can serve as the *front end* of early industrial design and later elaborate or export into engineering representations.

15 Discussion

15.1 Why the signature is a design object

The most consequential design decision in BDL is treating the function signature as visible product intent. In programming, a signature is often documentation and a static contract around code. In BDL it can precede any code-like definition and remain useful on its own.

The statement

$$?f : \text{Tilt} \rightarrow \text{Brightness}$$

says that the designer has already committed to a causal design relationship and to its semantic boundary. It does **not** say how the mapping is computed. This is exactly the kind of partial commitment common in concept design. The type system can therefore support design exploration rather than merely police a finished implementation.

15.2 Nominal identity in space and in time

BDL makes the same choice twice, on two axes, and the symmetry is not accidental.

On the axis of quantity, $\text{Sem } [n, d]$ makes semantic identity nominal. Two quantities of equal dimension are not interchangeable, equality is decided by name, and moving between them requires an explicit constructor. On the axis of time, $\text{Signal } [d]\tau$ makes temporal identity nominal. Two signals in different domains cannot be combined, equality is decided by name, and moving between them requires an explicit synchronization operator. Under this correspondence hold is the temporal counterpart of mk_n : the operator that crosses a boundary the type system otherwise keeps closed, and that must appear in the design when it is crossed.

Both axes refuse implicit coercion, both turn conversion into a visible artifact rather than a compiler action, and both are decided by syntactic equality, so the kernel needs one comparison mechanism rather than two.

This is where BDL departs from the tradition it otherwise draws on. Synchronous dataflow languages also treat clocks as types, and then recover them by inference, which is appropriate for their setting: programs are textual, users are control engineers, and annotating every clock would overwhelm the source. BDL's setting differs in three ways. Programs are graphs, in which every edge is already a positioned object, so annotation is nearly free. The relevant notion is a periodic rate rather than a Boolean subclock, which requires much less machinery. And the information needed to resolve a clock question does not arrive until realization binding, so inference postpones each decision to the moment at which the designer is least equipped to make it, whereas declaration raises it while the question is still semantic.

Stated generally: inference is well suited to recovering structure the author did not wish to state. Here the structure in question is the design.

15.3 Why not make everything a formula?

A symbolic engine is valuable for mappings because many product relationships are naturally mathematical and because formulas remove procedural intermediate variables. However, not all product behavior is a pure function. Temporal qualification, discrete events, history, mode-dependent behavior, and external actions require state and effects. BDL therefore uses formulas aggressively where they fit but does not pretend that a CAS can replace a reactive semantics.

15.4 Why not make StateHandler a general algebraic handler immediately?

General algebraic handlers are powerful, but their semantic machinery is larger than the initial design problem requires. Introducing

continuation semantics without a corresponding user need would complicate the kernel and proof burden while adding no surface capability. The current request-policy model captures the desired separation between intent and realization. A future extension can strengthen it if use cases reveal a genuine need for resumable or scoped operations.

15.5 Formal limits

The current calculus assumes discrete logical clocks, deterministic primitives, and finite temporal state. It does not yet define continuous dynamics, probabilistic sensor estimates, distributed clock uncertainty, or hybrid-system semantics. These are substantial extensions, not footnotes.

A nearer limitation concerns actions. An action is currently a request issued and forgotten: the request signature contains only an operation name and parameter type, and nothing in the calculus represents acknowledgement, failure, or asynchronous progress. Physical actuators do all three. A motor stalls, exceeds its travel, or fails to answer within a deadline, and a design that cannot express the difference between a command issued and a command completed cannot express a large class of ordinary product behavior. This is not a deferred elegance but the first thing a real device will demand. A likely extension is to pair a request operation with a typed asynchronous outcome event, making the temporal relation between issue and completion explicit rather than pretending that completion is a synchronous return value.

Likewise, the metatheory in this paper is a precise design target, not a completed mechanization. Before a PL venue should accept strong claims of soundness, the core calculus should be mechanized in Lean, Coq, Agda, or another proof assistant, including preservation for temporal state, row-policy transformations, and graph evaluation.

16 Research Roadmap

A realistic implementation should proceed in deliberately narrow stages.

Stage 0: representational feasibility. Before any implementation, re-express the behavior of an existing, completed device in BDL notation on paper, including its actuator arbitration and its timing constraints. If the central relationships cannot be stated, or are stated more verbosely than the firmware they replace, the hypothesis is wrong and the cost of learning this is a few days rather than a few months. The five-axis printer above is the intended subject.

Stage 1: typed mapping editor. Implement semantic types, signature-first Mapping Blocks, formulas/curves, typed holes, signal mapping, and basic unit checking. Use a small library of abstract sensors and actuators. This stage is large enough that it should carry an internal checkpoint: a single mapping authored as a signature, checked, and then given a definition, end to end, before the remaining authoring modes are built.

Stage 2: temporal behavior. Add *rise*, *for*, *after*, *while*, *local history*, and nested *StateHandlers* with *reset-on-entry* semantics.

Stage 3: realization. Bind domains to concrete rates on a constrained hardware profile, expose explicit synchronization, and generate a small runtime for microcontrollers or a simulator.

Stage 4: action policies and validation. Add actuator conflict analysis, safety policies, range analysis, and latency obligations.

Stage 5: empirical evaluation. Compare against statecharts and node-based embedded prototyping with industrial-design users, then deploy in a real studio course or industry project.

The important restraint is that BDL should not begin by attempting to replace CAD, EDA, MBSE, PLC languages, and simulation software simultaneously. Its first scientific contribution should be

the representation and interaction model for executable behavioral intent.

17 Conclusion

Modern industrial products increasingly combine physical form with sensing, computation, and control, yet designers still lack a behavior medium with the immediacy that CAD provides for geometry. BDL proposes that the missing medium should not be a friendlier version of procedural programming. It should be a language in which semantic product relationships are first-class design artifacts.

The core mechanism is signature-first authoring. A mapping such as `Tilt -> Brightness` exists and can be checked before its formula exists. Typed holes preserve intentional incompleteness; formulas and curves refine relationships; signals and events represent temporal values without callbacks; StateHandlers scope behavior without turning every design into a statechart; and action requests separate product intent from device realization. A small typed kernel then supplies dimensions, declared clock domains, causality, deterministic tick semantics, and explicit validation obligations. Temporal identity is authored on the same footing as semantic identity: in both cases the type system records a distinction the designer has drawn, and crossing it is a visible act rather than an inferred one.

The research question is therefore not whether designers can be taught a simpler programming language. It is whether product behavior can become a *design material* whose structure is intuitive at the surface and rigorous underneath. If that succeeds, designers can express and validate a larger portion of a product's functional logic while it is still a design problem, rather than discovering it only after it has become an engineering problem.

References

- [1] S. Greenberg and C. Fitchett, "Phidgets: Easy Development of Physical Interfaces through Physical Widgets," in *Proceedings of the 14th Annual ACM Symposium on User Interface Software and Technology*, ACM, 2001, pp. 209–218. doi: 10.1145/502348.502388.
- [2] B. Hartmann *et al.*, "Reflective Physical Prototyping through Integrated Design, Test, and Analysis," in *Proceedings of the 19th Annual ACM Symposium on User Interface Software and Technology*, ACM, 2006, pp. 299–308. doi: 10.1145/1166253.1166300.
- [3] T. R. G. Green and M. Petre, "Usability Analysis of Visual Programming Environments: A 'Cognitive Dimensions' Framework," *Journal of Visual Languages & Computing*, vol. 7, no. 2, pp. 131–174, 1996, doi: 10.1006/jvlc.1996.0009.
- [4] C. Elliott and P. Hudak, "Functional Reactive Animation," in *Proceedings of the Second ACM SIGPLAN International Conference on Functional Programming*, ACM, 1997, pp. 263–273. doi: 10.1145/258948.258973.
- [5] G. H. Cooper and S. Krishnamurthi, "Embedding Dynamic Dataflow in a Call-by-Value Language," in *Programming Languages and Systems: 15th European Symposium on Programming*, Springer, 2006, pp. 294–308.
- [6] D. Harel, "Statecharts: A Visual Formalism for Complex Systems," *Science of Computer Programming*, vol. 8, no. 3, pp. 231–274, 1987, doi: 10.1016/0167-6423(87)90035-9.
- [7] G. D. Plotkin and M. Pretnar, "Handling Algebraic Effects," *Logical Methods in Computer Science*, vol. 9, no. 4, 2013, doi: 10.2168/LMCS-9(4:23)2013.
- [8] C. Omar, I. Voysey, M. Hilton, J. Aldrich, and M. A. Hammer, "Hazelnut: A Bidirectionally Typed Structure Editor Calculus," in *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages*, ACM, 2017, pp. 86–99. doi: 10.1145/3009837.3009900.
- [9] J.-L. Colaço and M. Pouzet, "Clocks as First Class Abstract Types," in *Proceedings of the Third ACM International Conference on Embedded Software*, ACM, 2003.
- [10] Z. Yang, M. Paviotti, N. Wu, B. van den Berg, and T. Schrijvers, "Structured Handling of Scoped Effects," *Proceedings of the ACM on Programming Languages*, vol. 6, no. ICFP, 2022, doi: 10.1145/3547654.
- [11] J.-L. Colaço, B. Pagano, and M. Pouzet, "A Conservative Extension of Synchronous Data-flow with State Machines," in *Proceedings of the 5th ACM International Conference on Embedded Software*, ACM, 2005. doi: 10.1145/1086228.1086261.
- [12] B. Hartmann, L. Abdulla, M. Mittal, and S. R. Klemmer, "Authoring Sensor-Based Interactions by Demonstration with Direct Manipulation and Pattern Recognition," in *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, ACM, 2007, pp. 145–154. doi: 10.1145/1240624.1240646.
- [13] National Instruments, "LabVIEW." 2024.
- [14] The MathWorks, "Simulink and Stateflow." 2024.
- [15] Modelica Association, "Modelica Language Specification, Version 3.6." 2023.
- [16] A. Kennedy, "Relational Parametricity and Units of Measure," in *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, ACM, 1997, pp. 442–455. doi: 10.1145/263699.263761.
- [17] T. Bourke and M. Pouzet, "Zélus: A Synchronous Language with ODEs," in *Proceedings of the 16th International Conference on Hybrid Systems: Computation and Control*, ACM, 2013, pp. 113–118. doi: 10.1145/2461328.2461348.
- [18] Object Management Group, "OMG Systems Modeling Language (SysML) Version 2.0." [Online]. Available: <https://www.omg.org/spec/SysML/2.0>